```
new_value)
    noexcept;
template<typename BaseType>
BaseType atomic_exchange(atomic<BaseType>* p, BaseType new_value)
noexcept;
```

Effects

```
return p->exchange(new_value);
```

Std::Atomic_Exchange_Explicit Nonmember Function

Atomically stores a new value in an `atomic<BaseType>` instance and reads the prior value.

Declaration

```
template<typename BaseType>
BaseType atomic_exchange_explicit(
    volatile atomic<BaseType>* p, BaseType new_value, memory_order
order)
    noexcept;
template<typename BaseType>
BaseType atomic_exchange_explicit(
    atomic<BaseType>* p, BaseType new_value, memory_order order)
noexcept;
```

Effects

```
return p->exchange(new_value,order);
```

Std::Atomic::Compare_Exchange_Strong Member Function

Atomically compares the value to an expected value and stores a new value if the values are equal. If the values aren’t equal, updates the expected value with the value read.

Declaration

```
bool compare_exchange_strong(
    BaseType& expected,BaseType new_value,
    memory_order order = std::memory_order_seq_cst) volatile
noexcept;
bool compare_exchange_strong(
```